

Day 26: Ring Buffers & Lock-free ISR Communication

Platform: nRF52840 | RTOS: Zephyr RTOS

Learning Objectives

K_MSGQ_DEFINE for ISR-to-Thread Messaging

Zephyr's `k_msgq` (message queue) is the **safest and most idiomatic** way to pass data from an ISR to a thread. It uses a fixed-size FIFO with atomic access:

```
struct sensor_data { uint16_t value; uint32_t ts; };
K_MSGQ_DEFINE(sensor_q, sizeof(struct sensor_data), 8, 4);

// In ISR (non-blocking):
struct sensor_data d = {.value = adc_val, .ts = k_uptime_get_32()};
k_msgq_put(&sensor_q, &d, K_NO_WAIT); // Drop if full

// In thread (blocking):
struct sensor_data d;
k_msgq_get(&sensor_q, &d, K_FOREVER); // Block until data available
```

k_fifo for Dynamic Messages

`k_fifo` is a linked-list FIFO that can hold items of varying sizes. Items must be allocated **before** being put in the FIFO:

```
K_FIFO_DEFINE(my_fifo);
struct item { void *fifo_reserved; uint8_t data[16]; };
// Pre-allocate from slab, put in fifo:
k_fifo_put(&my_fifo, item_ptr);
struct item *out = (struct item *)k_fifo_get(&my_fifo, K_FOREVER);
```

Ring Buffer (SPSC — Single Producer, Single Consumer)

On a **single-core MCU** like nRF52840, a SPSC ring buffer is inherently lock-free:

- The **producer** (ISR) writes to `head` — only it modifies `head`
- The **consumer** (thread) reads from `tail` — only it modifies `tail`
- No lock needed because each index is owned by exactly one writer

This works because Cortex-M4 word reads/writes are atomic (no torn reads of 32-bit aligned variables).

Zephyr ring_buf API

Zephyr's `ring_buf` is optimized for **byte streams** (e.g., UART RX):

```
RING_BUF_DECLARE(uart_rx_ring, 256); // 256-byte ring buffer

// Producer (UART ISR):
ring_buf_put(&uart_rx_ring, &byte, 1);

// Consumer (thread):
uint8_t buf[32];
uint32_t len = ring_buf_get(&uart_rx_ring, buf, sizeof(buf));
```

Lock-free Data Structures for ISR Communication

On single-core Cortex-M:

- `volatile` variables + aligned access = atomic on 32-bit values
- SPSC ring buffer = lock-free for one producer, one consumer
- `k_msgq` = ISR-safe, handles multi-producer/consumer with internal spinlock

On multi-core systems (like nRF5340 with M33+M33), you need explicit memory barriers (`__DMB()`) and atomic operations (`atomic_t`).

Memory Barriers in Embedded

A **memory barrier** prevents the CPU and compiler from reordering memory accesses across the barrier:

```
volatile uint32_t ready_flag = 0;
// Write data first, then set flag (order matters!)
memcpy(shared_buf, data, len);
__DMB();           // Data Memory Barrier – ensures memcpy completes before flag write
ready_flag = 1;    // ISR sees data only after this
```

Practice Tasks

1. Build a UART RX parser using `ring_buf` — detect `\r\n` terminated commands and signal a semaphore
 2. Implement a SPSC ring buffer in 20 lines of C++ — verify correctness with a test on host
 3. Use `K_MSGQ_DEFINE` to pass ADC samples from a timer ISR to a processing thread
 4. Measure message queue throughput: how many samples/second can you push through `k_msgq` before dropping?
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day26
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)

- [Nordic DevZone](#)
- Source: [src/day26/main.cpp](#)